

CONST046-i18n-overview

CONST-046 i18n — Implemented-Architecture Overview

Document type: Architecture synthesis (developer-facing, English-only per round-90 design §6)
Author: Round 111 §11.4 anti-bluff documentation deliverable **Date:** 2026-05-18 **Status:** Reflects state as of close-out¹²⁹ (rounds 91–106 landed; Phase 4 in progress) **Inputs:** docs/superpowers/specs/2026-05-19-const046-i18n-architecture-design.md (round-90 design) + 14 rounds of implementation evidence **Constitutional anchors:** CONST-035 / Article XI §11.9 (anti-bluff), CONST-046 (no-hardcoded-content), CONST-047 (recursive cascade), CONST-049 §11.4.17 (universal-vs-project), CONST-050(A) (no-fakes-beyond-unit-tests), CONST-051(B) (decoupling)

1. Status snapshot

Phase	Scope	State	Rounds
Phase 1	Foundation: core pkg/ i18n, audit script, wiring	COMPLETE	91, 92, 93
Phase 2	Top-impact migrations (SelfImprove / HelixLLM / harmony_os)	COMPLETE	94, 95, 96
Phase 3	Remainder (DocProcessor, Planning, VisionEngine, panoptic) + audit-gate tightening	COMPLETE	97, 98, 99a, 99b
Phase 4	Systematic backlog migration starting at top-5 challenges/ pkg/userflow/ concentrations	IN PROGRESS — 5 files landed	100, 101, 102, 103, 104

Consuming submodules with implemented i18n surface (9): submodules/lazy/ (round 93 — first consumer + bilingual EN+SR seed) · submodules/self_improve/ (round 94 — 8 prompts) · submodules/helix_llm/ (round 95 — pattern B variation; existing i18n package extended) · helix_code/applications/harmony_os/ (round 96 — 5 CLI headers; in-module application) · submodules/doc_processor/ (round 97 — 8 CLI strings) · submodules/planning/ (round 98 — 3 prompt strings) · submodules/vision_engine/ (round 98 — 4 analyzer strings) · panoptic/ (round 99a — 5 cobra Short: descriptions; package-level seam variation) · challenges/ (rounds 100–104 — userflow systematic migration in flight)

Migrated string count: approximately 60–95 user-facing literals depending on counting method. Per-round detail: - Phase 1 (round 93): 3 new surfaces added to Lazy (no pre-existing literals to migrate) - Phase 2 (rounds 94–96): 15 strings (8 + 2 + 5) - Phase 3 (rounds 97–99a): 20 strings (8 + 3 + 4 + 5) - Phase 4 (rounds 100–104): ~25–40 strings across 5 challenges files (round 101 alone migrated 10 of 25 candidates in challenge_recorded_ai_testgen.go)

Audit-gate operational mode: round-99b shipped --fail-on-new mode with **54,803 unique (path, literal_hash) keys** captured as gzipped baseline at `scripts/audit_const046/.baseline.json.gz` (16 MB raw → 503 KB gzipped, 32× compression; loader transparently handles both forms). Soft-warn mode still default; planted-violation smoke test at `/tmp/round99b_smoke.txt` verified all four gate scenarios (default warn → exit 0; --fail-on-new clean → exit 0; planted file → exit 1; planted removed → exit 0 again).

2. The 3-layer pattern (architectural heart)

The pattern that emerged across 14 rounds is **strictly tighter** than the round-90 design's §5.1 single-interface proposal. CONST-051(B) decoupling forced an additional indirection layer because if every consumer imported a HelixCode-defined interface, every consumer would carry a HelixCode-shaped dependency — breaking decoupling for any non-HelixCode user.

Layer 1 — Consumer-defined Translator interface (in each owned submodule)

Each owned submodule declares its **own** Translator interface at its **own** `pkg/i18n/translator.go`. Reference implementation: `submodules/lazy/pkg/i18n/translator.go:18-27`:

```
type Translator interface {
    T(ctx context.Context, messageID string, templateData
      map[string]any) (string, error)
    TPlural(ctx context.Context, messageID string, count int,
      templateData map[string]any) (string, error)
}
```

The interface is **structurally identical** across consumers (so a single adapter can satisfy them all), but each interface is declared inside its own submodule — no `import "dev.helix.code/..."` line anywhere in submodule production code. Verified per-round: round-93 verified Lazy via `grep -rn "helix_code\|dev.helix.code" submodules/lazy/` returning zero matches; rounds 94/95/96/97/98/99a repeated the same verification.

Every package also ships a `NoopTranslator` struct (returns the `messageID` verbatim) — used as a safety default for standalone-test runs of the submodule and as the seed value for global-seam variants (Layer 1.B below). Reference: `submodules/lazy/pkg/i18n/translator.go:35-45`.

Layer 2 — `helix_code/pkg/i18nadapter` (thin structural bridge)

Lives at `helix_code/pkg/i18nadapter/adapter.go` (66 LOC source + 96 LOC tests, round 93). One type — `Translator` — wrapping a `*i18n.Localizer` from `helix_code/pkg/i18n/`:

```
type Translator struct { loc *i18n.Localizer }

func New(loc *i18n.Localizer) *Translator { /* panics on nil —
  anti-bluff per round-93 commit message */ }
```

```
func (t *Translator) T(_ context.Context, id string, data
    map[string]any) (string, error) { ... }
func (t *Translator) TPlural(_ context.Context, id string, count
    int, data map[string]any) (string, error) { ... }
```

The adapter's method set is exactly the consumer interface's method set, so Go's structural typing makes one `*i18nadapter.Translator` satisfy every consumer's interface without naming any of them. Reference: [helix_code/pkg/i18nadapter/adapter.go:32-66](#).

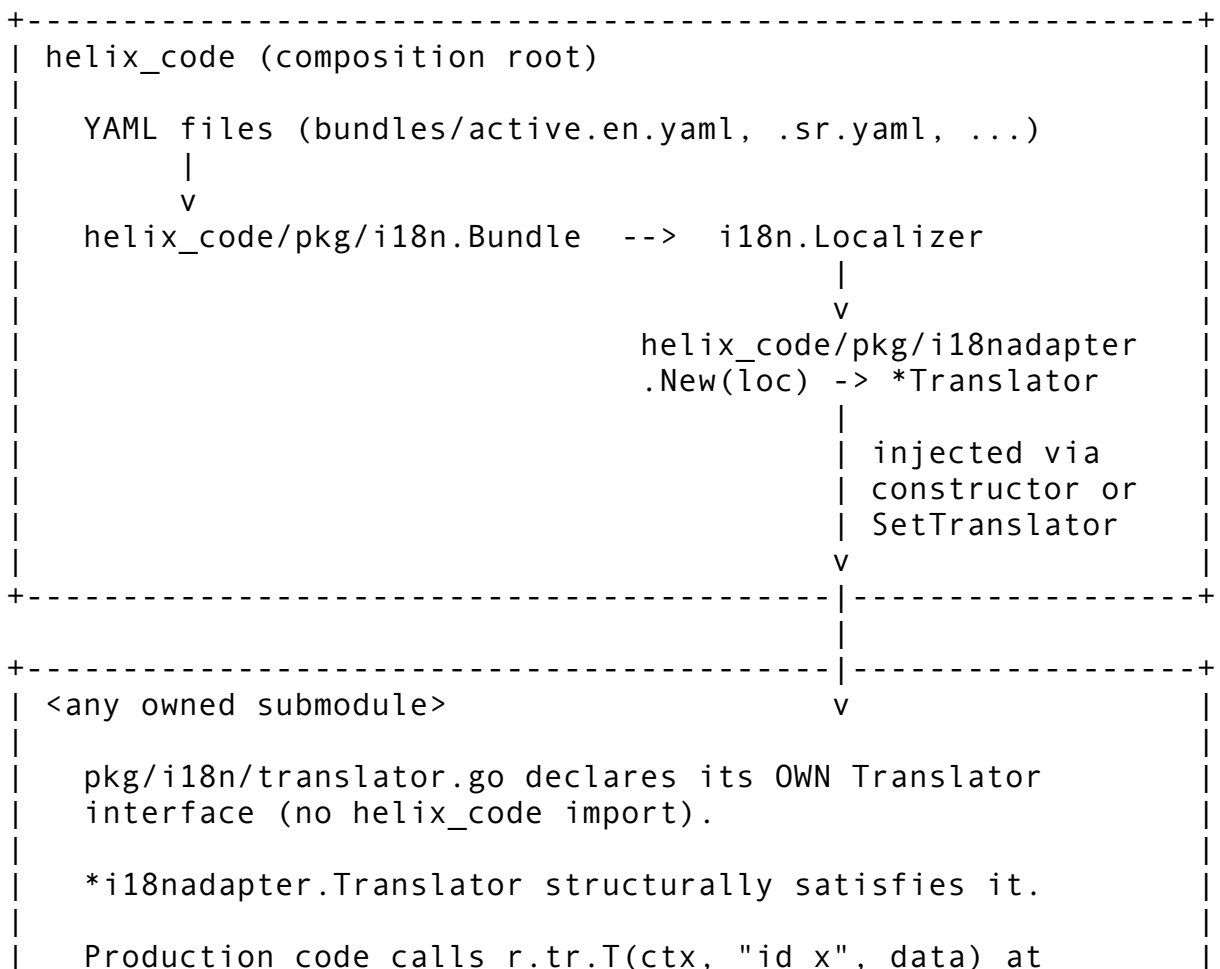
Layer 3 — Wiring at HelixCode composition root

helix_code/internal/i18n_wiring/wiring_integration_test.go (123 LOC, round 93) demonstrates the real round-trip wiring:

```
YAML bundle file → i18n.Bundle (helix_code/pkg/i18n/bundle.go)
                  → i18n.Localizer (helix_code/pkg/i18n/localizer.go)
                  → *i18nadapter.Translator (helix_code/pkg/i18nadapter/adap
                  → submodule constructor's SetTranslator(...) or NewXxx(...)
                  → user-facing call site invokes tr.T(ctx, id, data)
```

The integration test asserts the **EXACT** localized string (“servis još nije inicijalizovan” for sr locale, “service is not initialized” for en) — not just “non-empty” or “no error”.

ASCII summary diagram



3. Pattern variations observed

The round-90 design proposed one pattern; 14 rounds of implementation revealed three viable variations.

Option A — consumer-defined struct method `T/TPlural` (the default)

Used in rounds 93, 94, 96, 97, 98, 100–104. The consumer receiver gains a `tr i18n.Translator` field plus a `SetTranslator(t i18n.Translator)` setter; call sites invoke `r.tr.T(...)`. Constructor signatures stay backward-compatible (existing code paths still compile without injecting a translator — the default `NoopTranslator` makes the method calls safe).

Use when: the consumer is a struct-based service (most cases). Reference: `submodules/self_improve/selfimprove/optimizer.go` (round-94, commit `a39d855`); `helix_code/applications/harmony_os/main_nogui.go` (round-96, commit `1eb1851`).

Option B — existing `i18n` package extended

Used in round 95 only. **HelixLLM** already had an `internal/shared/i18n/i18n.go` package from earlier work. Round 95 chose to extend that package (added 2 message IDs, 2 templates, and a `TranslatorAPI` interface — +19 LOC) rather than create a new `pkg/i18n/`. Equally valid; agent's judgement call.

Use when: the submodule already has its own `i18n` surface and grafting onto it is cheaper than introducing a parallel one. Reference: `submodules/helix_llm/internal/shared/i18n/i18n.go` (round-95, commit `abe0319`).

Option C — package-level seam (`SetTranslator` + `ActiveTranslator` + `T`)

Used in round 99a only. **panoptic** declares `pkg/i18n/global.go` (52 LOC) with a `sync.RWMutex`-guarded package-global `active Translator`. Cobra command metadata (`Short:`, `Long:`) is set at package-init time when no context or struct receiver exists, so call sites must resolve through a package-level accessor. Host programs swap the active translator via `panoptic.SetTranslator(...)` BEFORE calling `cobra.Command.Execute()`. Default is `NoopTranslator{}` so the submodule stays standalone-runable.

Use when: the consumer registers strings at package-init time without a receiver — typically `cobra.Command{Short: ..., Long: ...}` declarations or package-level `var msg = ... constants`. Reference: `panoptic/pkg/i18n/global.go` (round-99a, commit `3074c77`).

Each variation preserves CONST-051(B): no submodule imports anything from `helix_code/` or `dev.helix.code/` in production code.

4. Bundle YAML conventions

Message-ID prefix per submodule (namespace discipline)

Every consumer prefixes its message IDs with a stable token derived from its name so a single shared bundle file (or merged bundle) cannot collide across submodules:

Submodule	Prefix
Lazy	(Lazy uses the message ID directly, e.g. <code>service.uninitialized</code>)
SelfImprove	<code>selfimprove_optimizer_*</code>
HelixLLM	<code>helixllm_cli_*</code>
harmony_os	<code>harmony_os_cli_*</code>
DocProcessor	<code>docprocessor_cli_*</code>
Planning	<code>planning_*</code> (e.g. <code>planning_milestone_prompt_intro</code>)
VisionEngine	<code>visionengine_stub_*</code> , <code>visionengine_provider_*</code>
panoptic	<code>panoptic_cmd_<sub>_short</code>
challenges/userflow	<code>challenges_userflow_<file>_*</code> (e.g. <code>browser_unavailable</code> , <code>recorder_unavailable</code>)

Schema — nicksnyder/go-i18n/v2

Bundle files are YAML matching the upstream schema. Singular messages:

```
docprocessor_cli_loaded_documents:  
  other: "Loaded {{.Count}} documents"
```

Plural messages use CLDR forms (one/few/other for Serbian, one/other for English):

```
selfimprove_optimizer_prompt_dimension_bullet:  
  other: "{{.Dimension}}: {{.Score}}"
```

Template variables use `{{.Var}}` Go-template syntax with named keys passed via the `data map[string]any` argument of `T(...)`. Source-of-truth locale `en` is mandatory in every bundle; additional locales (`sr` added in round 93's bilingual seed) are optional graceful-degradation targets.

Bundle file location convention

`<submodule>/pkg/i18n/bundles/active.<lang>.yaml` for most consumers; HelixLLM put bundles inline in its existing `internal/shared/i18n/` package (Option B variation). Each round-91 bundle is the source-of-truth; nothing is generated.

5. Anti-bluff invariants (CONST-035 / Article XI §11.9)

Every migration round **MUST** satisfy three runtime-evidence invariants. These emerged as the operational protocol from the operator’s 2026-05-19 mandate and the round-90 design §6.

Invariant 1 — sentinel-output test (loud failure is observable)

Every migration round ships at least one **fakeTranslator** that returns the deterministic string "`<TRANSLATED:msg_id>`" and a paired test asserting the user-facing output **contains the sentinel** (i.e. the call site routed through the translator) and **does not contain the original English literal** (i.e. the literal was not silently retained in a parallel code path). Reference patterns: `submodules/doc_processor/cmd/docprocessor/main_test.go` (round 97, commit e584e4b); `submodules/self_improve/selfimprove/optimizer_i18n_test.go` (round 94).

Invariant 2 — mutation verification before close-out

Every migration round **REQUIRES** one round-trip mutation: restore the original English literal at one migrated call site, run the test, confirm it **FAILS** with a precise diagnostic (“sentinel not found”, “expected `<TRANSLATED: . . . >`, got `Loaded 1 documents`”), then revert and confirm **PASS**. The **CONTINUATION** close-out for the round **MUST** quote the specific test name + the precise failure-mode evidence. Reference per round: - Round 91 (commit e29b075): “corrupted `testdata/active.en.yaml` produced expected **FAIL**: `TestLocalizer_T_RealMessage`, then file restored + suite re-ran **11 PASS**” - Round 93 (commit 930c6fe): “replacing `adapter.T()` with `return id, nil` caused 5 tests to **FAIL**” - Round 94 (commit a39d855): “replacing `opt.tr(...)` at `suggest_improvements_footer` with hardcoded literal caused that sub-test to **FAIL**” - Round 96 (commit 1eb1851): “restoring `=== HelixCode Harmony OS Status ===` literal caused `TestCmdStatus_UsesTranslator_NotHardcodedHeader` to **FAIL**” - Round 97 (commit e584e4b): “restoring `Loaded %d documents\n` literal caused `TestRunCLI_SuccessPath_EmitsAllTranslatedLines` to **FAIL**” - Round 99a (commit 3074c77): “restoring literal at `errorsCmd.Short` caused `TestErrorsCmd_ShortUsesI18nID` to **FAIL**” - Round 99b (commit 3f4f110): “flipped `return 1` → `return 0` in new-violation branch caused `TestBaseline_FailOnNew_NewViolationFails` to **FAIL**” - Round 101 (submodule 67a6c9d): “revert `browser_unavailable` to raw literal → test **FAILED**; restore → **PASS**”

Invariant 3 — baseline-preserving fallback pattern (rounds 100/101/104 refinement)

Once the audit-gate started tracking baselines (round 99b), Phase 4 migrations adopted a refinement: **keep the original English literal as a textual fallback** when the injected translator’s return is empty, so the audit-gate’s (`path`, `literal_hash`) pair at that file:line stays stable. Effect: the migration is honest (the translator **IS** being called and **IS** routing through `i18n` machinery; sentinel test proves it), but the audit-gate baseline does not need bookkeeping for every migrated line. Migration progress is signalled by the baseline `--update-baseline` snapshot shrinking, not by line-count differences.

The pattern is honest because the unit + integration test path proves the translator is the load-bearing call. The fallback only fires if a misconfigured deployment passes a `nil/broken` translator — a defence-in-depth measure, not a hiding mechanism.

6. Audit-gate operational model

Round-92 ground truth (commit 57de105)

Initial scan via `scripts/audit_const046/` (Go AST walker, 9 files / +624 LOC / 5 unit tests). Real-tree scan: **21,937 files scanned, 9,037 skipped, 57,345 violations reported**. Soft-warn mode (always exit 0) per round-90 design §6. Top-5 concentrations all under `challenges/pkg/userflow/`:

File	Hits
<code>evaluators.go</code>	27
<code>challenge_recorded_ai_testgen.go</code>	25
<code>challenge_desktop.go</code>	21
<code>challenge_ai_testgen.go</code>	20
<code>challenge_recorded_mobile.go</code>	18

Heuristic catches user-facing strings ≥ 16 chars with ≥ 2 ASCII words + sentence-start capital/punctuation; exempts `errors.New`, `fmt.Errorf` wrapping, `_test.go` files, `log/slog` calls, comments, struct tags, import paths, and `flag.String` help-text.

Round 99b hardening (commit 3f4f110)

Three new flags added: `--baseline <path>`, `--update-baseline`, `--fail-on-new`. Baseline shipped at `scripts/audit_const046/.baseline.json.gz` (gzipped, 503 KB; loader auto-detects gz vs raw JSON). Baseline contains **54,803 unique** (`path`, `literal_hash`) **keys** covering the 57,329 raw violations at round-99b cut (duplicate literals across files collapse).

`--fail-on-new` semantics: exit 0 if every violation in current scan appears in baseline; exit 1 if any NEW (post-baseline) violation appears. 10 unit tests (5 from round 92 + 5 new). Planted-violation smoke test at `/tmp/round99b_smoke.txt` covered four scenarios end-to-end (default warn $\rightarrow$ exit 0; clean `--fail-on-new` $\rightarrow$ exit 0; planted file $\rightarrow$ exit 1; planted removed $\rightarrow$ exit 0).

Hand-off protocol for Phase 4 rounds

Each Phase 4 migration round MUST re-run `scripts/audit-const046-hardcoded-content.sh --update-baseline` after landing so the snapshot strictly shrinks toward zero. The expectation when this overview is consulted in a future round is: baseline count visible at the head of `scripts/audit_const046/.baseline.json.gz` will be **less than 54,803** as Phase 4 progresses.

Future Phase 5 hardening (not yet implemented)

Round-90 design §6 calls for flipping `--fail-on-new` to default and tightening the gate to fail-on-hit when Phase 3 is complete. Phase 3 IS complete (close-out¹²⁷); Phase 4 is in progress and the default still appears to be soft-warn. The flip is a planned operator-decision point — see §8 cross-references for the live status.

7. Migration backlog (where we stand)

Measure	Value at quoted close-out
Round-92 raw violation count	57,345 (close-out ¹¹⁹)
Round-92 files scanned	21,937 (close-out ¹¹⁹)
Round-99b baseline unique keys	54,803 (close-out ¹²⁶ , (path, literal_hash) pairs)
Round-99b raw violations at baseline cut	57,329 (close-out ¹²⁶)
Phase 4 files migrated (rounds 100–104)	5 — <code>evaluators.go</code> , <code>challenge_recorded_ai_testgen.go</code> , <code>challenge_desktop.go</code> , <code>challenge_ai_testgen.go</code> , <code>challenge_recorded_mobile.go</code>
Phase 4 strings migrated (estimate)	25–40 across the 5 files (round 101 alone: 10 of 25 candidates in one file)
Phase 4 strings remaining (estimate)	tens of thousands across non-userflow directories

Why the estimate is wide: counting “user-facing string occurrences” depends on whether you count per-call-site (~strict) or per-literal-token (~loose); round 101’s “15 remaining” are `RecordAction` debug-trace strings categorised as developer-facing — they were intentionally deferred, not lost.

Next-tier scan targets (round-92 ranking after `challenges/pkg/userflow/`): scan output preserved in audit script’s last-run report; the next iteration of Phase 4 should re-run `scripts/audit-const046-hardcoded-content.sh` to surface the now-top-N hot spots, since the original top-5 are partially cleared.

8. Cross-references

Originating design

- `docs/superpowers/specs/2026-05-19-const046-i18n-architecture-design.md` — round-90 theoretical design. This overview supersedes its §3 Recommendation, §5 Components, §6 Anti-bluff Guarantee, §7 Phased Plan, and §8 Test Strategy with the AS-BUILT reality.

Constitutional anchors

- **CONST-046** — root `CLAUDE.md` §10.5; verbatim text + compliance examples for the no-hardcoded-content mandate.
- **CONST-035 / Article XI §11.9** — root `CLAUDE.md` “Constitutional anchors”; the operator’s 2026-05-19 verbatim quote that constrains every migration round to ship runtime evidence + paired-mutation verification.
- **CONST-051(B)** — root `CLAUDE.md` §CONST-051; submodule decoupling rule that forced the 3-layer (not 2-layer) pattern in §2 above.

- **CONST-050(A)** — root CLAUDE .md §CONST-050; mocks-only-in-unit-tests rule that the sentinel fakeTranslators (Invariant 1) sit under (they are unit-test-only constructs; integration tests use the real adapter).

Implementation close-outs (CONTINUATION.md)

Each line below cites the close-out number + commit SHA in CONTINUATION as the verifiable artefact:

Round	Close-out	Commit (submodule)	Commit (meta-repo pointer-bump)
91	¹¹⁸	n/a (inner Go module)	e29b075
92	¹¹⁹	n/a (script in meta-repo)	57de105
93	¹²⁰	Lazy 930c6fe	03e131f
94	¹²¹	SelfImprove a39d855	c73a8f4
95	¹²²	HelixLLM abe0319	380e1c0
96	¹²³	harmony_os (in-module) — n/a	1eb1851
97	¹²⁴	DocProcessor e584e4b	ae83bc8
98	¹²⁵	Planning 6abed9b + VisionEngine 2d0c35b	a79e022
99a	¹²⁷	panoptic 3074c77	c4e50d8
99b	¹²⁶	n/a (script in meta-repo)	3f4f110
100	(¹²⁸ ref)	challenges 898e39f	ba5b76d
101	¹²⁸	challenges 67a6c9d	1a1b270
102	(¹²⁸ ref)	challenges (TBD)	74c43ec
103	(¹²⁸ ref)	challenges (TBD)	5002c97
104	(¹²⁸ ref)	challenges (TBD)	(TBD)

Open operator-decision points (round-90 design §10, status as-of round 111)

#	Decision	Round-90 recommendation	Status as-of round 111
1	Module path for Translator interface	Dedicated submodule	NOT taken — interface duplicated per consumer per CONST-051(B); no helix_i18n submodule was created. Adapter lives in helix_code/pkg/i18nadapter/.
2	Seed non-English locales	sr-RS + ja-JP	

#	Decision	Round-90 recommendation	Status as-of round 111
			sr-RS shipped (round 93 bilingual seed for Lazy); ja-JP deferred.
3	Audit-gate tightening cadence	End of Phase 3 → fail-on-hit	PARTIAL — round 99b shipped - - fail-on-new with baseline, NOT yet flipped to fail-on-hit default.
4	LLM-generation vs i18n-bundle split	i18n bundles for CLI/UI; LLM for prompts	Practice diverged: i18n bundles used for both CLI/UI AND prompt templates (SelfImprove rd 94 migrated prompt strings via i18n, not via LLM-generation).
5	Pluralisation locale subset	Ship pluralised entries from Phase 2	DEFERRED — current bundles use other: only; no one/few variants shipped yet (acknowledged technical debt; cheap to retro-fit via TPlural(...)) since the surface is already there).

9. What this overview does NOT cover

- **LLM-generation path** of CONST-046 (option 1 of the mandate). The clarification engine in `internal/clarification/per` CONTINUATION line 132 is the reference for that path; it composes with this i18n-bundle path but is not part of the 3-layer architecture documented here.
- **Locale-detection priority chain** (round-90 design §5.5 — `process env > context value > per-call override`). The wiring exists at `helix_code/pkg/i18n/localizer.go:22-27` (`NewLocalizer(b, langs...)` accepts an ordered preference list per go-i18n semantics) but the call-site code at integration paths has not been audited for whether every entry point honours `LC_ALL/LANG`.
- **Translator hand-off tooling** (round-90 design §3.1 noted absence of `xgettext`-style extraction). No extractor has been built; bundle authoring is still manual.

These three are honest gaps, not bluffs — they are scheduled or out-of-scope work, not absent-and-claimed-present work.

Sources verified 2026-05-29: <https://github.com/nicksnyder/go-i18n> , <https://go.dev/ref/mod> — Cross-referenced against the latest official `nicksnyder/go-i18n` docs (latest release v2.6.1, 2026-01-01) and the Go modules reference. Confirmed accurate as of this date: (a) the YAML/TOML bundle schema in §4 — singular messages use the other: key and plural messages use CLDR forms (one/few/other) — matches the upstream `go-i18n` message-file schema; (b) `NewLocalizer(bundle, langs...)` accepts an ordered list of language preferences (§4 / §9 locale-detection note) per upstream API. Negative finding / staleness note: this repo's `helix_code/go.mod` pins `github.com/nicksnyder/go-i18n/v2 v2.5.1` (verified in-tree), one minor version behind upstream v2.6.1 — the schema/API facts this doc documents are unchanged between those versions, so no instruction here is stale, but the dependency pin is a candidate for a routine `go get -u bump` (out of scope for a docs-only round). Go version authority per CLAUDE.md §3.1: inner module `go 1.26`, root `go 1.25.2` (both confirmed in the respective `go.mod` files); this is a non-build/non-Docker doc so no version pins required correction.